1 — Python fundamentals for Data Science (beginner → practical)

These are the Python basics you'll *actually* use in data work.

Variables & data types

- `int`, `float`, `str`, `bool` — basic primitives.
- `list` — ordered, mutable sequences; used for small collections.
- `tuple` — ordered, immutable.
- `dict` — key → value mapping (like a spreadsheet row).
- `set` — unordered unique elements.

Example:

```
a = 10          # int
b = 3.14        # float
name = "Vivek" # str
is_good = True # bool

l = [1,2,3]
t = (1,2,3)
d = {"name": "Vivek", "age": 20}
```

Why this matters: Libraries like Pandas will convert lists/dicts into arrays/tables — so understanding these helps you convert raw data into formats those libraries expect.

Control flow (if/for/while) & functions

Example:

```
for i in range(3):
    print(i)

def add(a,b):
    return a+b
print(add(4,5))
```

Why: You'll loop through rows, write helper functions to clean columns, and wrap repetitive processing in functions.

Comprehensions & generators

Fast way to build lists/dicts:

```
squares = [x*x for x in range(5)]   # [0,1,4,9,16]
even = {x for x in range(10) if x%2==0}
```

Why: Data cleaning often maps or filters columns — comprehensions are concise and fast.

Packages & environment

- Install packages with `pip` or prefer `conda` (Anaconda) for data-science stacks.
- Virtual environments: `venv` or `conda env` — keep project dependencies isolated.

Command examples:

```
pip install numpy pandas matplotlib seaborn scikit-learn
# or with conda:
conda create -n ds python=3.10
conda activate ds
conda install numpy pandas matplotlib seaborn scikit-learn
```

Why: Different projects need different versions. Use envs to avoid version conflicts.

---

2 — NumPy (Numerical Python) — the foundation

NumPy provides an efficient, C-backed array object (`ndarray`) and fast mathematical operations (vectorized). NumPy arrays are central because Pandas builds on top of them.

Key concepts

- `ndarray` — n-dimensional array (1D, 2D, …). Homogeneous type (all numbers typically).

- Vectorized operations — apply operations element-wise without Python loops (fast).
- Broadcasting — rules for operating on arrays of different shapes.
- Indexing & slicing — like lists but more powerful.
- Universal functions (`np.mean`, `np.sum`, etc.)

Examples

```
import numpy as np

a = np.array([1,2,3])         # 1D
b = np.array([[1,2],[3,4]])  # 2D matrix

# Vectorized ops
print(a + 10)     # [11,12,13]
print(a * 2)      # [2,4,6]

# Elementwise math
print(np.sqrt(a)) # [1.,1.4142...]

# Create arrays
zeros = np.zeros((3,2))
ones = np.ones(5)
rand = np.random.randn(4,3)  # normal distribution

# Mean, sum, axis
m = np.mean(b, axis=0)  # mean of each column
```

Broadcasting example

```
A = np.array([[1,2,3],[4,5,6]])  # shape (2,3)
v = np.array([1,0,1])            # shape (3,)
print(A + v)  # v is broadcast to each row
```

Common uses in data science

- Fast numeric computations (vectorized math).
- Converting lists to arrays for ML libraries.
- Linear algebra (matrices) for algorithms.

Pitfalls

- NumPy arrays require homogeneous types — mixing strings breaks numeric operations.
- Be careful with shapes when using matrix ops — transpose (`.T`) matters.

Practice

```
# Create a 5x5 array of random numbers and compute mean of each row
import numpy as np
X = np.random.rand(5,5)
row_means = X.mean(axis=1)
print(row_means)
```

---

3 — Pandas — Structured data (Series & DataFrame)

Pandas is the *workhorse* for tabular data (CSV, Excel, SQL exports). Two main objects:

- `Series` — 1D labeled array (like a single column).
- `DataFrame` — 2D table with rows and columns (like a spreadsheet / SQL table).

Why Pandas?

- Easy to read/write CSV, Excel, JSON.
- Powerful indexing, grouping, merging.
- Ideal for cleaning, transforming, exploring data.

Basic usage

```
import pandas as pd

# From dict
data = {'Name': ['A','B','C'], 'Age':[20,21,19]}
df = pd.DataFrame(data)

# Read CSV
df = pd.read_csv('data.csv')

# Quick views
df.head()     # first 5 rows
df.tail()
df.shape      # (rows, cols)
df.info()     # column types + non-null counts
df.describe() # numeric summary (mean, std, quartiles)
```

Column access & manipulation

```
# Access columns
ages = df['Age']
df['Age2'] = df['Age'] * 2
```

```
# Filter rows
df[df['Age'] > 20]

# Select subset
df.loc[:, ['Name','Age']]   # by label
df.iloc[0:3, 0:2]           # by position
```

Missing values

```
df.isnull().sum()        # count nulls per column
df.dropna()              # remove rows with any null
df.fillna(0)             # fill nulls with 0
df['col'].fillna(df['col'].mean(), inplace=True)
```

GroupBy & aggregation

```
df.groupby('Category')['Sales'].sum()
df.groupby(['A','B']).agg({'C':'mean', 'D':'sum'})
```

Merging and joins

```
pd.merge(df1, df2, on='id', how='inner')  # inner/left/right/outer
join
```

Reshaping: pivot, melt

```
pivot = df.pivot_table(index='date', columns='product',
values='sales', aggfunc='sum')
melted = df.melt(id_vars=['id'], value_vars=['v1','v2'])
```

Datetime handling

```
df['date'] = pd.to_datetime(df['date_str'])
df.set_index('date', inplace=True)
df.resample('M').sum()  # monthly resample
```

Categorical data & encoding

- Use `astype('category')` for strings with few unique values to save memory.
- For ML: one-hot encoding `pd.get_dummies()` or `sklearn.preprocessing.OneHotEncoder`.

Example end-to-end small dataset

```
import pandas as pd
```

```
df = pd.DataFrame({
    'city': ['Pune','Delhi','Pune','Delhi','Mumbai'],
    'sales': [100,200,150,None,80]
})
# Fill missing
df['sales'] = df['sales'].fillna(df['sales'].mean())
# Group
df.groupby('city')['sales'].sum()
```

Pitfalls

- Chained indexing: `df[df['a']>0]['b'] = 1` — this can give warnings; use `.loc[...]` instead.
- Modifying a copy vs the original — use `inplace` or assign back.

Practice

Load any CSV (or make one), compute:

- number of missing values per column,
- mean of numeric columns,
- top 3 rows with highest value in a chosen column.

---

4 — Data visualization (Matplotlib & Seaborn)

Visualization is critical for understanding data.

Matplotlib (low-level plotting)

```
import matplotlib.pyplot as plt
x = [1,2,3,4]
y = [10,20,15,25]
plt.plot(x,y, marker='o')
plt.title('Line plot')
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

Common plots:

- `plt.plot()` — line plot
- `plt.scatter()` — scatter plot

- `plt.bar()` — bar chart
- `plt.hist()` — histogram

Seaborn (statistical high-level)

Seaborn sits on Matplotlib and gives attractive default styles + convenient statistical plots.

```
import seaborn as sns
sns.histplot(data=df, x='age', kde=True)
sns.barplot(x='city', y='sales', data=df)
sns.boxplot(x='category', y='value', data=df)
sns.heatmap(df.corr(), annot=True)
```

When to use which

- Use Matplotlib for custom, low-level control.
- Use Seaborn for quick, beautiful statistical plots.

Tips for plotting

- Always label axes and title plot.
- Use `plt.figure(figsize=(w,h))` for size.
- For multiple plots, use `plt.subplot()`.

Practice

- Plot histogram of a numeric column.
- Make a scatter plot of two numeric columns and color points by a categorical column.

---

5 — Scikit-learn (the ML toolbox)

Scikit-learn (sklearn) offers simple, uniform APIs for many ML tasks: classification, regression, clustering, preprocessing.

Typical ML workflow

1. Load and clean data (Pandas).
2. Split features `X` and target `y`.
3. Train/test split.

4. Preprocess features (scale, encode).
5. Choose model, train (`fit`).
6. Predict on test set (`predict`).
7. Evaluate.

Example: Linear Regression

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Suppose df has 'area' and 'price'
X = df[['area']]
y = df['price']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("MSE:", mean_squared_error(y_test, y_pred))
```

Classification example: Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

model = LogisticRegression()
model.fit(X_train, y_train)      # X_train: features, y_train:
labels (0/1)
y_pred = model.predict(X_test)
print(accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Preprocessing

- Scaling: `StandardScaler`, `MinMaxScaler`
- Encoding categorical: `OneHotEncoder`, `OrdinalEncoder`
- Pipelines: chain preprocessing + model

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([('scaler', StandardScaler()), ('clf',
LogisticRegression())])
pipe.fit(X_train, y_train)
```

Model selection & validation

- `train_test_split` — simple split.
- `cross_val_score` — k-fold CV.
- `GridSearchCV` — hyperparameter search.

Example:

```
from sklearn.model_selection import GridSearchCV
params = {'C':[0.1,1,10]}
grid = GridSearchCV(LogisticRegression(), params, cv=5)
grid.fit(X_train, y_train)
print(grid.best_params_)
```

Metrics

- Regression: MSE, RMSE, MAE, R^2.
- Classification: accuracy, precision, recall, F1-score, ROC-AUC.
- Clustering: silhouette score.

Pitfalls

- Data leakage: never scale or encode using the full dataset before splitting.
- Imbalanced classes: accuracy can be misleading; use precision/recall/F1.
- Overfitting: complex models can fit noise — use cross-validation and regularization.

Practice

- Build logistic regression on Iris dataset (from sklearn.datasets import load_iris). Report accuracy and classification report.

---

6 — Other important libraries & tools (short, but essential)

SciPy

- Builds on NumPy. Provides optimization, signal processing, stats functions.

```
from scipy import stats
```

```
stats.ttest_ind(a,b)
```

Statsmodels

- For statistical models with detailed summaries (OLS regression with p-values).

```
import statsmodels.api as sm
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print(model.summary())
```

XGBoost / LightGBM / CatBoost

- High-performance gradient boosting libraries for structured/tabular data (used in competitions).
- Syntax similar to scikit-learn or native API; often tuned via hyperparameters.

Plotly / Bokeh

- Interactive plots for dashboards and exploratory analysis.

TensorFlow / PyTorch

- Deep learning frameworks. Start with scikit-learn first; deep learning only when needed.

Jupyter Notebook / Colab

- Interactive environment for code + notes + plots. Colab gives free GPU/TPU — great to experiment.

---

7 — Typical real-world data science workflow (step by step)

1. **Define problem & target.**
2. **Gather data** (CSV, DB, APIs).
3. **Explore** with `df.head()`, `df.describe()`, visualizations.
4. **Clean**: missing values, duplicates, wrong types.
5. **Feature engineering**: create useful features (dates → day/month/year, text → length).

6. **Encode categorical variables** (one-hot or ordinal).
7. **Split** into train/test.
8. **Scale** numerical features if needed.
9. **Train & validate** models, tune hyperparameters.
10. **Evaluate** with appropriate metrics.
11. **Deploy / report**: save model, make dashboard, report conclusions.

---

8 — Worked walkthrough (mini project) — predicting house prices

I'll give a compact, fully runnable example you can copy into a Jupyter cell. It shows Pandas → feature engineering → sklearn.

```
# 1. imports
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# 2. toy data (replace with pd.read_csv('file.csv') in real life)
df = pd.DataFrame({
    'area': [50, 70, 80, 120, 60, 75],
    'bedrooms': [1,2,2,3,1,2],
    'age': [10,5,15,20,2,7],
    'price': [2000,3000,3200,4500,2100,3100]  # target
})

# 3. basic explore
print(df.head())
print(df.describe())

# 4. features and target
X = df[['area', 'bedrooms', 'age']]
y = df['price']

# 5. split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# 6. model
model = LinearRegression()
model.fit(X_train, y_train)

# 7. predict + evaluate
y_pred = model.predict(X_test)
print("MSE:", mean_squared_error(y_test, y_pred))
print("Coefficients:", model.coef_)
```

```
print("Intercept:", model.intercept_)
```

Explainable: coefficients show the expected change in price per unit change in a feature (holding others constant).

---

9 — Common beginner mistakes & how to avoid them

1. **Not exploring the data first** — always plot and inspect distributions.
2. **Leaking data from test into train** — don't do preprocessing on the whole dataset before splitting.
3. **Ignoring missing data** — blindly dropping rows can bias results.
4. **Using accuracy for imbalanced classification** — use precision/recall/F1.
5. **Not scaling when needed** — some algorithms (SVM, KNN) need scaling.
6. **Trying too many features without feature selection** — more features can mean more noise.

---

10 — Quick cheat-sheet (commands you'll use daily)

- `pd.read_csv('file.csv')`, `df.head()`, `df.info()`, `df.describe()`
- `df.isnull().sum()` → check missing
- `df.dropna()`, `df.fillna()`
- `df.groupby('col')['val'].agg(['mean','sum'])`
- `pd.merge(df1, df2, on='id')`
- `sklearn.model_selection.train_test_split`
- `scaler = StandardScaler(); X_train = scaler.fit_transform(X_train); X_test = scaler.transform(X_test)`
- `model.fit(X_train, y_train); y_pred = model.predict(X_test)`

---

11 — Short exercises (do these now)

1. Create a Pandas DataFrame with 100 random rows: `age` (18–60), `salary` (random), `city` chosen from 3 cities. Then:
    - Show the average salary per city (groupby).
    - Plot salary distribution.
2. Load the Iris dataset (`from sklearn.datasets import load_iris`). Build a logistic regression classifier and report accuracy.
3. Take any CSV (or create one), intentionally add some `NaN`s, then practice filling them using mean/median/mode and compare results.

---

12 — Final learning plan (for a beginner who wants to become productive)

Week 1: Python basics + practice small scripts + environment setup (Anaconda/Jupyter)
Week 2: NumPy & Pandas fundamentals + small data cleaning tasks
Week 3: Visualization (Matplotlib & Seaborn) + exploratory data analysis (EDA) projects
Week 4: Intro to Machine Learning (scikit-learn): regression + classification + evaluation
Week 5+: Advanced: feature engineering, model tuning, tree-based models (XGBoost), basics of deep learning (PyTorch/TensorFlow) if needed